STAT 432 Homework 10

Conner Jensen (connerj2)

Contents

Question 1: Bias-variance trade-off **1**

(a) . . . 3

(b) . . . 4

Question 2: MNIST 1, 6, 7 **4**

(a) . . . 5

(b) . . . 5

(c) . . . 7

Question 3: Planning a random-forest skill **8**

(a) . . . 8

(b) . . . 8

Question 1: Bias-variance trade-off

Five predictors are $N(0,1)$ with pairwise correlation 0.3. The mean function is $f(\mathbf{x}) = x_1 + x_3 + x_5$. I generate 200 test $\mathbf{x}$ once and keep them fixed. Each of 100 repetitions draws a new training set of size 200. Forests use 500 trees, bootstrap samples of size 200 (`resample.prob = 1`, `replacement = TRUE`), and `reinforcement = FALSE`. Seed `432101`.

```
set.seed(432101)
Sig <- matrix(0.3, 5, 5)
diag(Sig) <- 1
Rchol <- chol(Sig)
genX <- function(n) matrix(rnorm(n * 5), n, 5) %*% Rchol
fmean <- function(X) X[, 1] + X[, 3] + X[, 5]

n <- 200
nsim <- 100
Xte <- genX(n)
fte <- fmean(Xte)
nmins <- c(1, 5, 10, 20, 30)
mtrys <- 1:5
pred_nmin <- array(NA_real_, c(nsim, n, length(nmins)))
pred_mtry <- array(NA_real_, c(nsim, n, length(mtrys)))
```

```r
fit_rf <- function(X, y, mtry, nmin) {
  quiet_rlt(X, y, model = "regression", ntrees = 500,
      mtry = mtry, nmin = nmin,
      resample.prob = 1, replacement = TRUE,
      reinforcement = FALSE, importance = FALSE,
      print.summary = 0, use.cores = 1)
}

for (s in seq_len(nsim)) {
  Xtr <- genX(n)
  ytr <- fmean(Xtr) + rnorm(n)
  for (j in seq_along(nmins)) {
    pred_nmin[s, , j] <- predict(
      fit_rf(Xtr, ytr, mtry = 3, nmin = nmins[j]), Xte
    )$Prediction
  }
  for (j in seq_along(mtrys)) {
    pred_mtry[s, , j] <- predict(
      fit_rf(Xtr, ytr, mtry = mtrys[j], nmin = 10), Xte
    )$Prediction
  }
}

avg_bv <- function(pred) {
  mbar <- colMeans(pred)
  bias2 <- (mbar - fte)^2
  vari <- apply(pred, 2, function(z) mean((z - mean(z))^2))
  c(sq_bias = mean(bias2), variance = mean(vari))
}
sn <- t(sapply(seq_along(nmins), function(j) avg_bv(pred_nmin[, , j])))
sm <- t(sapply(seq_along(mtrys), function(j) avg_bv(pred_mtry[, , j])))
data.frame(nmin = nmins, round(sn, 3))
```

```
##   nmin sq_bias variance
## 1    1   0.330    0.114
## 2    5   0.335    0.102
## 3   10   0.376    0.082
## 4   20   0.494    0.064
## 5   30   0.624    0.055
```

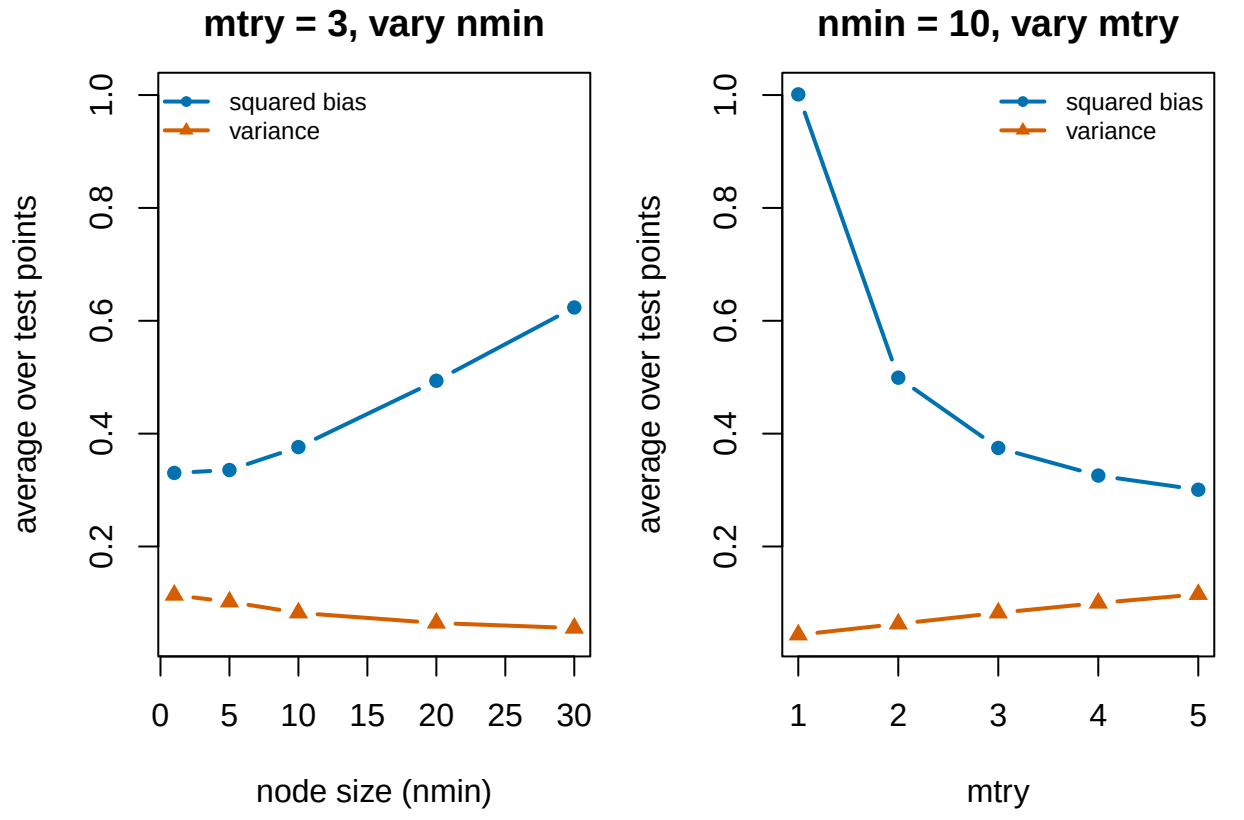
```r
data.frame(mtry = mtrys, round(sm, 3))
```

```
##   mtry sq_bias variance
## 1    1   1.001    0.043
## 2    2   0.499    0.063
## 3    3   0.375    0.082
## 4    4   0.326    0.100
## 5    5   0.301    0.115
```

```r
par(mfrow = c(1, 2), mar = c(4.2, 4.2, 2.6, 0.8))
yl <- range(sn, sm)
plot(nmins, sn[, "sq_bias"], type = "b", lwd = 2, pch = 16,
```

```
     col = "#0072B2", ylim = yl, xlab = "node size (nmin)",
     ylab = "average over test points",
     main = "mtry = 3, vary nmin")
lines(nmins, sn[, "variance"], type = "b", lwd = 2, pch = 17,
      col = "#D55E00")
legend("topleft", bty = "n", cex = 0.75,
       legend = c("squared bias", "variance"),
       col = c("#0072B2", "#D55E00"), pch = c(16, 17), lwd = 2)

plot(mtrys, sm[, "sq_bias"], type = "b", lwd = 2, pch = 16,
     col = "#0072B2", ylim = yl, xlab = "mtry",
     ylab = "average over test points",
     main = "nmin = 10, vary mtry")
lines(mtrys, sm[, "variance"], type = "b", lwd = 2, pch = 17,
      col = "#D55E00")
legend("topright", bty = "n", cex = 0.75,
       legend = c("squared bias", "variance"),
       col = c("#0072B2", "#D55E00"), pch = c(16, 17), lwd = 2)
```



(a)

Squared bias and variance are estimated at each of the 200 fixed test points from the 100 forest predictions, then averaged. Bias is versus $f(\mathbf{x})$, not versus Y.

With `mtry = 3`, moving `nmin` from 1 to 30 raises squared bias from 0.330 to 0.624 and drops variance from

0.114 to 0.055. With `nmin = 10`, moving `mtry` from 1 to 5 drops squared bias from 1.001 to 0.301 and raises variance from 0.043 to 0.115.

(b)

A node size of 30 stops splitting while leaves are still pretty big. The forest is smoother and more stable, but it cannot follow $x_1 + x_3 + x_5$ as closely, so most of the error is bias. In this example that trade is not worth it: the extra bias is larger than the variance you save.

A large `mtry` (here 4 or 5) lets each split see the three signal variables more often, so bias falls a lot. The cost is that the trees look more alike, so the forest variance goes up. MSE still improves because the bias drop is bigger than the variance increase. `mtry = 1` is the other extreme: trees waste splits on X_2 and X_4 and the squared bias is about 1.

If each tree sampled 100 observations instead of 200, the same `nmin = 30` would be a much larger share of the bootstrap sample, so trees would stop even earlier and bias would get worse. To keep a similar depth I would move node size down. So the preferred `nmin` should shift toward smaller values.

Question 2: MNIST 1, 6, 7

Training uses digits $1, 6, 7$ from the first 1,000 rows ($n = 327$: 116, 94, 117). Test uses the same digits from the last 1,000 ($n = 317$: 104, 106, 107). PCA is fit on the training pixels, centered and not scaled. The first 20 PCs are the predictors. Forests have 1,000 trees and a bootstrap of size n. Seed `432102` for the default fit and the tuning grid; seed `432103` for the selected forest.

```
mn <- read.csv(gzfile("data/mnist-first2000.csv.gz"))
pix <- as.matrix(mn[, names(mn) != "Digit"])
lab <- mn$Digit
tr_rows <- which(lab[1:1000] %in% c(1, 6, 7))
te_rows <- 1000 + which(lab[1001:2000] %in% c(1, 6, 7))
ytr <- factor(lab[tr_rows], levels = c(1, 6, 7))
yte <- factor(lab[te_rows], levels = c(1, 6, 7))

cen <- colMeans(pix[tr_rows, ])
pc <- prcomp(sweep(pix[tr_rows, ], 2, cen, "-"),
             center = FALSE, scale. = FALSE)
Xtr <- pc$x[, 1:20]
Xte <- sweep(pix[te_rows, ], 2, cen, "-") %*% pc$rotation[, 1:20]
colnames(Xtr) <- colnames(Xte) <- paste0("PC", 1:20)
c(n_train = nrow(Xtr), n_test = nrow(Xte))
```

```
## n_train  n_test
##     327     317
```

```
table(ytr)
```

```
## ytr
##   1   6   7
## 116  94 117
```

```r
table(yte)
```

```
## yte
##   1   6   7
## 104 106 107
```

(a)

RLT defaults here are `mtry = max(1, 20/3) = 6` and `nmin = max(1, log n) = 5`. Forest OOB error is the majority vote over trees that did not use that observation.

```r
set.seed(432102)
fit_def <- quiet_rlt(Xtr, ytr, model = "classification", ntrees = 1000,
               resample.prob = 1, replacement = TRUE,
               reinforcement = FALSE, importance = FALSE,
               track.obs = TRUE, print.summary = 0, use.cores = 1)
c(mtry = fit_def$mtry, nmin = fit_def$nmin,
  oob = round(oob_class(fit_def, Xtr, ytr), 4))
```

```
##   mtry   nmin    oob
## 6.0000 5.0000 0.0336
```

Default OOB misclassification is 0.0336 (11/327).

(b)

I used `mtry` in $\{2, 4, 6, 10, 20\}$ and `nmin` in $\{1, 5, 10, 20\}$. The lowest OOB rate is `mtry = 2`, `nmin = 1` (0.0183). Smaller `mtry` and smaller leaves did better than the default on this grid. That matches Question 1 in a way: extra randomness (small `mtry`) plus deeper trees helped more than growing almost-bagged trees on all 20 PCs.

```r
set.seed(432102)
grid <- expand.grid(mtry = c(2, 4, 6, 10, 20),
                    nmin = c(1, 5, 10, 20))
grid$oob <- NA_real_
for (i in seq_len(nrow(grid))) {
  fit <- quiet_rlt(Xtr, ytr, model = "classification", ntrees = 1000,
              mtry = grid$mtry[i], nmin = grid$nmin[i],
              resample.prob = 1, replacement = TRUE,
              reinforcement = FALSE, importance = FALSE,
              track.obs = TRUE, print.summary = 0, use.cores = 1)
  grid$oob[i] <- oob_class(fit, Xtr, ytr)
}
grid[order(grid$nmin, grid$mtry), ]
```

```
##    mtry nmin        oob
## 1     2    1 0.01834862
## 2     4    1 0.02140673
## 3     6    1 0.02446483
## 4    10    1 0.03975535
```

```
## 5    20    1 0.03669725
## 6     2    5 0.02752294
## 7     4    5 0.03058104
## 8     6    5 0.03058104
## 9    10    5 0.03363914
## 10   20    5 0.03363914
## 11    2   10 0.02752294
## 12    4   10 0.03669725
## 13    6   10 0.03363914
## 14   10   10 0.04587156
## 15   20   10 0.04281346
## 16    2   20 0.03363914
## 17    4   20 0.03058104
## 18    6   20 0.04587156
## 19   10   20 0.04281346
## 20   20   20 0.04587156
```

```
ibest <- which.min(grid$oob)
grid[ibest, ]
```

```
##   mtry nmin        oob
## 1    2    1 0.01834862
```

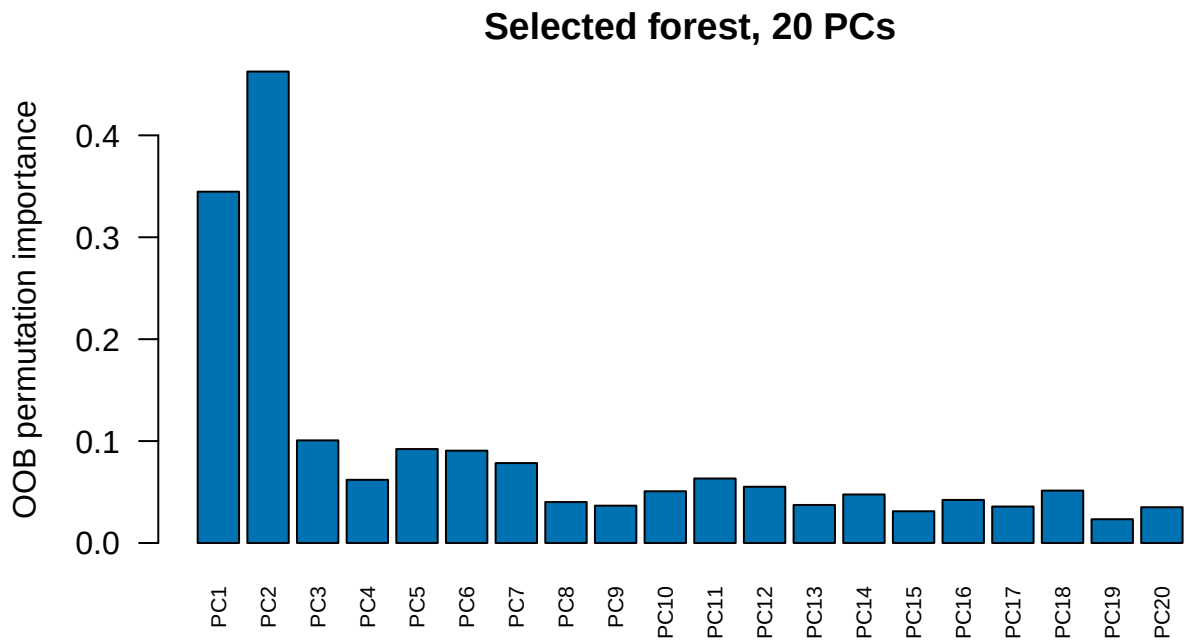
```
set.seed(432103)
fit_best <- quiet_rlt(Xtr, ytr, model = "classification", ntrees = 1000,
                mtry = grid$mtry[ibest], nmin = grid$nmin[ibest],
                resample.prob = 1, replacement = TRUE,
                reinforcement = FALSE, importance = TRUE,
                track.obs = TRUE, npermute = 1,
                print.summary = 0, use.cores = 1)
oob_sel <- oob_class(fit_best, Xtr, ytr)
vi <- as.numeric(fit_best$VarImp)
names(vi) <- colnames(Xtr)
round(oob_sel, 4)
```

```
## [1] 0.0275
```

```
round(vi, 3)
```

```
##   PC1   PC2   PC3   PC4   PC5   PC6   PC7   PC8   PC9  PC10  PC11  PC12  PC13
## 0.345 0.463 0.101 0.062 0.092 0.091 0.078 0.040 0.037 0.051 0.063 0.055 0.037
##  PC14  PC15  PC16  PC17  PC18  PC19  PC20
## 0.048 0.031 0.042 0.036 0.051 0.023 0.035
```

```
par(mar = c(4.2, 4.2, 2.4, 0.6))
barplot(vi, las = 2, cex.names = 0.7, col = "#0072B2",
        ylab = "OOB permutation importance",
        main = "Selected forest, 20 PCs")
```



I refit the selected setting with seed `432103` so importance and the test predictions come from one forest. That forest's OOB rate is 0.0275 (9/327). Permutation importance is highest on `PC2` and `PC1`, then a drop to `PC3`/`PC5`. The later PCs are small and pretty flat. So the first couple of components carry most of the 1 vs 6 vs 7 separation, which is what I would expect after PCA on these digits.

(c)

```
pred <- map_pred(fit_best, Xte)
mean(pred != yte)
```

```
## [1] 0.01577287
```

```
table(observed = yte, predicted = pred)
```

```
##          predicted
## observed   1   6   7
##        1 101   0   3
##        6   0 106   0
##        7   1   1 105
```

Test misclassification is 0.0158 (5/317). Three 1s are called 7, one 7 is called 1, and one 7 is called 6. Test labels were not used to pick `mtry` or `nmin`.

Question 3: Planning a random-forest skill

(a)

I would keep the skill short. A decent model already knows how to grow a forest, compute OOB, and make a confusion matrix. If I write a long lecture it just repeats itself, and on Homework 8 a long skill still left the seed and the cutoff unspecified. I would only write the choices I actually care about: use RLT with reinforcement off, tune `mtry` and node size from OOB, and do not touch the test labels until the end. Standard steps can stay as one-liners. Personal defaults are the part that is worth writing down. I am not making a `SKILL.md` for this question.

(b)

1. Split off a test set first. Do not use those labels for any tuning.
2. Fit ordinary random forests (reinforcement off) with a bootstrap sample of size n.
3. Compare settings with OOB error, not test error.
4. Tune `mtry` and node size together on a small grid.
5. After you pick a setting, refit once and report that forest's OOB rate.
6. Report test error and a confusion matrix only for the selected model.
7. Include permutation variable importance for the selected forest.
8. Use the RLT package in R unless I say otherwise.
9. Keep the report short: one grid table and one importance figure, no extra methods.
10. If I give a seed, use it. For pixel data, center but do not scale before PCA.